

Multilevel penguins

Nicole Cruz

Last updated: 15 September, 2025

Packages

```
library(tidyverse)    # Graphs with ggplot()
library(GGally)       # Extensions to ggplot
library(knitr)        # Tables with kable()
library(reshape2)     # data reshaping between wide & long
library(ggokabeito)   # colorblind friendly colors
library(viridis)      # colorblind friendly colors
library(brms)         # Hierarchical Bayesian models
library(rstan)        # Interface to Stan
library(shinystan)    # Model output graphs
library(bayesplot)    # Model output graphs

# General settings
options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)
options(contrasts = c("contr.equalprior_pairs", "contr.poly"))
set.seed(1234) # For reproducibility
```

Research question

Can a penguins' flipper length be predicted from their bill length? To what extent does this vary between species, and/or between islands?

Overview of the pattern in the data

```
g1 <- ggplot(penguins, aes(x = bill_len, y = flipper_len))+
  geom_point(alpha = .5, aes(shape = species, color = species))+
  scale_color_viridis_d(begin = .15, end = .85)+
  scale_fill_viridis_d(begin = .15, end = .85)+
  stat_ellipse(geom = "polygon", alpha = 0.25, aes(color = species, fill = species))+
  theme_bw()+
  labs(x = "Bill length", y = "Flipper length")
g1
```

Initial priors

```
# What are the defaults?
default_prior(flipper_len ~ bill_len + (1 | species)
  , data = penguins
  , family = gaussian()
)

##           prior      class      coef  group resp dpar nlpar lb ub
##           (flat)         b
##           (flat)         b  bill_len
```

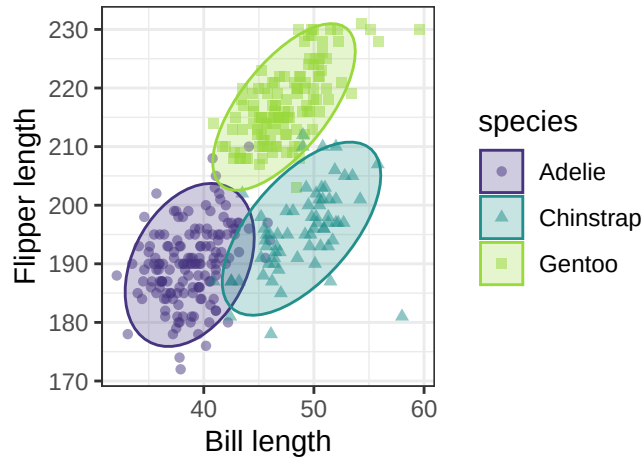


Figure 1: Flipper length as a function of bill length for penguins living in the Palmer archipel.

```
## student_t(3, 197, 16.3) Intercept
## student_t(3, 0, 16.3) sd 0
## student_t(3, 0, 16.3) sd species 0
## student_t(3, 0, 16.3) sd Intercept species 0
## student_t(3, 0, 16.3) sigma 0
## source
## default
## (vectorized)
## default
## default
## (vectorized)
## (vectorized)
## default
```

The model has default priors for the following coefficients:

- `bill_len` (class `b`, for the fixed effect of bill length): an improper flat prior over the reals.
- `Intercept` (class `Intercept`, for the fixed effect of the intercept after centering all other predictors around zero): a `student_t` prior that is weakly informative (a property it obtains by “looking” at the data).
- `Intercept species` (class `sd`, for the random intercept of species): a `student_t` prior that is weakly informative and has a lower bound at zero, since standard deviations cannot be negative.
- `sigma` (class `sigma`, for the residual standard deviation after taking all other parameters into account): a `student_t` prior that is weakly informative and has a lower bound at zero.

Own priors - version 1

Lets try to improve on the defaults by replacing the flat prior on `b` with a more informative one

```
prior_1 <- c(prior(normal(0, 5), class = b, coef = bill_len))
```

Try it out

```
output <- capture.output(
  m_prior_1 <- brm(flipper_len ~ bill_len + (1 | species)
    , data = penguins
    , family = gaussian()
    , prior = prior_1
    , sample_prior = "only" # "TRUE"
```

```

, warmup = 1000
, iter = 4000
, chains = 4
, cores = 4
, save_pars = save_pars(all = TRUE)) # required to compute BFs
)
# We can double-check which priors were used:
# get_prior(m_prior_1)

set.seed(1234)

# Prior predictive check for the model overall:
pp_check(m_prior_1)

```

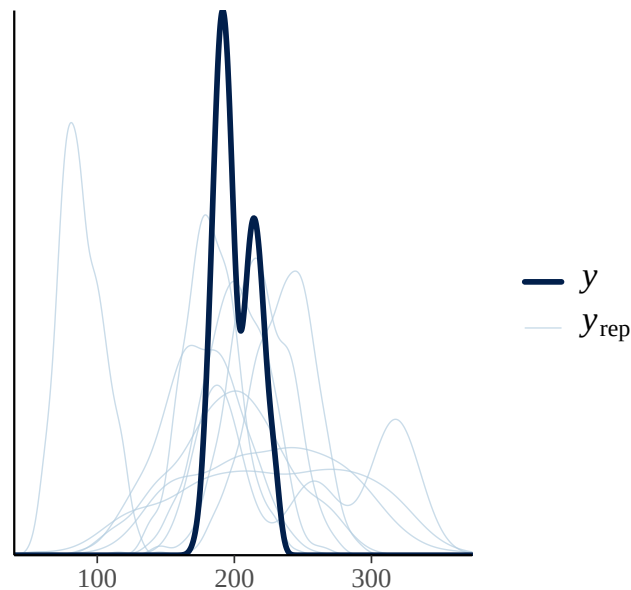


Figure 2: Prior predictive check 1.

The priors lead to reasonable predictions overall, though also seem to vary a lot. Consider replacing the t-distributions with normal distributions, whose narrower tails may reduce the overall variation somewhat.

Own priors - version 2

```

prior_2 <- c(prior(normal(0, 5), class = b, coef = bill_len),
             prior(normal(0, 5), class = sd, group = "species"))

```

Try it out

```

output <- capture.output(
m_prior_2 <- brm(flipper_len ~ bill_len + (1 | species)
, data = penguins
, family = gaussian()
, prior = prior_2
, sample_prior = "only"
, warmup = 1000
, iter = 4000
, chains = 4
, cores = 4

```

```

    , save_pars = save_pars(all = TRUE)) # required to compute BFs
)

set.seed(1234)

# Prior predictive check:
pp_check(m_prior_2)

```

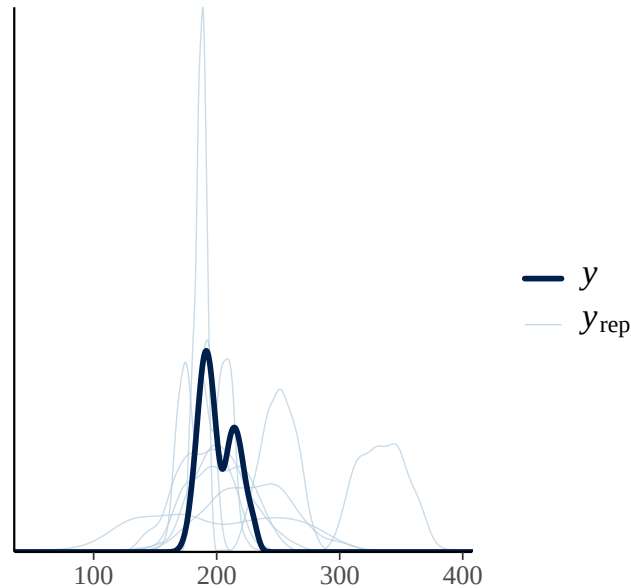


Figure 3: Prior predictive check 2.

```

# pp_check(m_prior_2, type='stat', stat='mean')

```

The correspondence between the simulated data y and the prior predictions y_{rep} looks a bit better than before. Let's now use these priors in a model that draws samples from the actual data.

Model 1

```

output <- capture.output(
m1 <- brm(flipper_len ~ bill_len + (1 | species)
  , data = penguins
  , family = gaussian()
  , prior = prior_2
  , sample_prior = TRUE
  , warmup = 1000
  , iter = 4000
  , chains = 4
  , cores = 4
  , save_pars = save_pars(all = TRUE)) # required to compute BFs
)

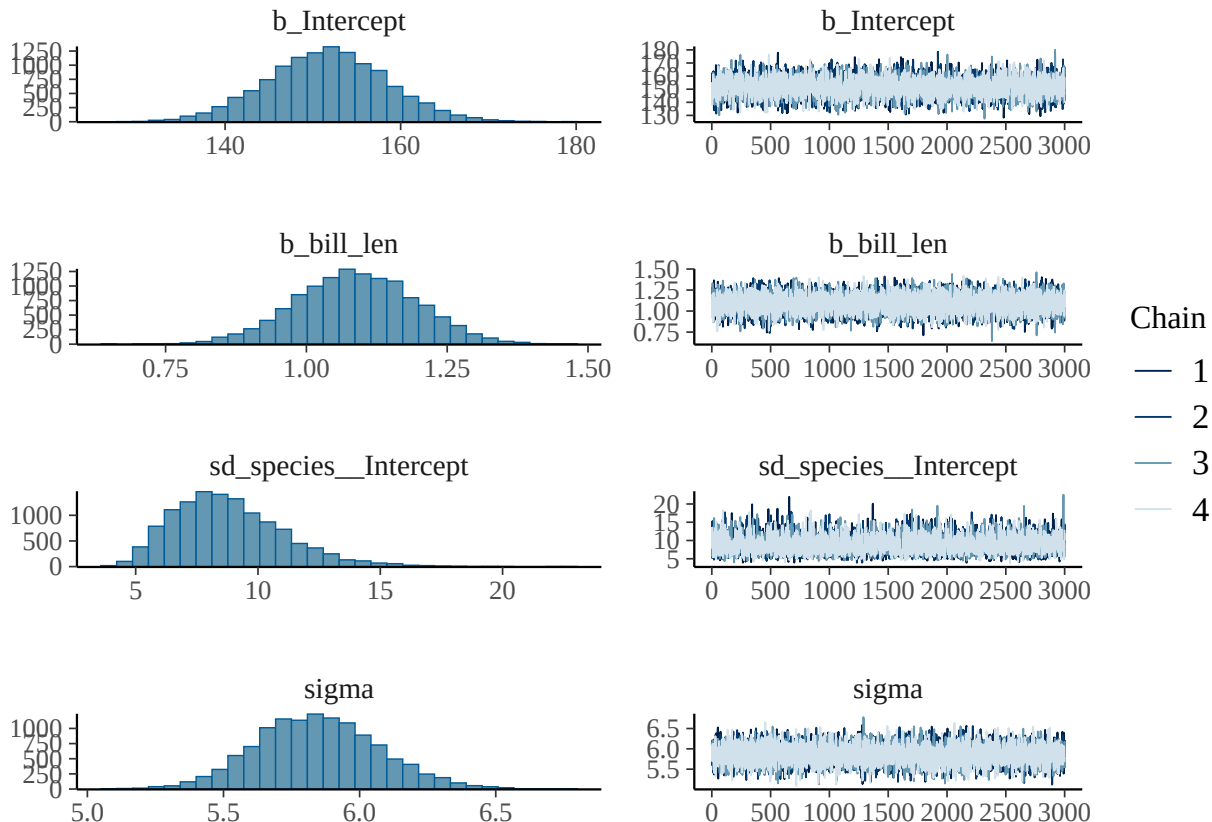
# Summary statistics for the model output,
# and convergence diagnostics
summary(m1, waic = TRUE)

## Family: gaussian
## Links: mu = identity; sigma = identity
## Formula: flipper_len ~ bill_len + (1 | species)

```

```
## Data: penguins (Number of observations: 342)
## Draws: 4 chains, each with iter = 4000; warmup = 1000; thin = 1;
## total post-warmup draws = 12000
##
## Multilevel Hyperparameters:
## ~species (Number of levels: 3)
## Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(Intercept)      8.79      2.26      5.24     14.02 1.00      5021      6552
##
## Regression Coefficients:
## Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept    151.64      6.85    138.22    165.19 1.00      5587      5905
## bill_len      1.09      0.11      0.88      1.29 1.00      8828      7036
##
## Further Distributional Parameters:
## Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sigma       5.85      0.23      5.42      6.31 1.00      8956      7727
##
## Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
plot(m1)
```



```
m1$fit
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=4000; warmup=1000; thin=1;
## post-warmup draws per chain=3000, total post-warmup draws=12000.
```

```
##
##               mean se_mean   sd    2.5%    25%
## b_Intercept   151.64    0.09  6.85   138.22   147.08
## b_bill_len     1.09    0.00  0.11    0.88    1.02
## sd_species__Intercept  8.79    0.03  2.26    5.24    7.13
## sigma         5.85    0.00  0.23    5.42    5.69
## Intercept     199.45    0.08  4.83   189.82   196.37
## r_species[Adelie,Intercept] -3.89    0.08  4.90   -13.76   -7.07
## r_species[Chinstrap,Intercept] -8.89    0.08  4.87   -18.47  -12.05
## r_species[Gentoo,Intercept]  13.78    0.08  4.84    4.22   10.61
## prior_Intercept  196.77    0.24 26.62   144.34   184.45
## prior_b_bill_len    0.03    0.05  5.04   -9.65   -3.34
## prior_sigma       18.14    0.21 22.77    0.57    5.62
## prior_sd_species    4.01    0.03  3.00    0.16    1.62
## lprior          -13.08    0.01  0.91   -15.41  -13.50
## lp__           -1103.11    0.03  1.93 -1107.80 -1104.16
## z_1[1,1]        -0.47    0.01  0.57   -1.60   -0.86
## z_1[1,2]        -1.08    0.01  0.62   -2.33   -1.49
## z_1[1,3]         1.67    0.01  0.68    0.41    1.19
##               50%    75%    97.5% n_eff Rhat
## b_Intercept   151.69   156.18   165.19  5593    1
## b_bill_len     1.09    1.16    1.29  8845    1
## sd_species__Intercept  8.51   10.14   14.02  5226    1
## sigma         5.84    5.99    6.31  8910    1
## Intercept     199.49   202.60   209.00  3691    1
## r_species[Adelie,Intercept] -3.90   -0.72    5.88  3799    1
## r_species[Chinstrap,Intercept] -8.90   -5.80    0.85  3683    1
## r_species[Gentoo,Intercept]  13.74   16.85   23.56  3693    1
## prior_Intercept  197.02   209.12   246.84 12417    1
## prior_b_bill_len   -0.02    3.44    9.92 11901    1
## prior_sigma       12.39   23.61   68.20 11909    1
## prior_sd_species    3.41    5.81   11.09 12064    1
## lprior          -12.88  -12.43  -11.95  5434    1
## lp__           -1102.80 -1101.68 -1100.36 4015    1
## z_1[1,1]        -0.46   -0.08    0.60  3895    1
## z_1[1,2]        -1.07   -0.66    0.08  3887    1
## z_1[1,3]         1.64    2.11    3.06  4146    1
##
## Samples were drawn using NUTS(diag_e) at Mon Sep 15 13:36:25 2025.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).

# See also: launch_shinystan(m1)
# The coefficients for fixed and random effects
# can also be extracted separately with:
# fixef(m1)
# raneef(m1)
```

We can inspect, and potentially modify, the underlying Stan code of a model with:

```
m3_stan <- make_stancode(data = penguins
  , formula = flipper_len ~ bill_len + (1 | species)
  , save_pars = save_pars(all=TRUE))
m3_stan
```

```

## // generated with brms 2.22.0
## functions {
## }
## data {
##   int<lower=1> N; // total number of observations
##   vector[N] Y; // response variable
##   int<lower=1> K; // number of population-level effects
##   matrix[N, K] X; // population-level design matrix
##   int<lower=1> Kc; // number of population-level effects after centering
##   // data for group-level effects of ID 1
##   int<lower=1> N_1; // number of grouping levels
##   int<lower=1> M_1; // number of coefficients per level
##   array[N] int<lower=1> J_1; // grouping indicator per observation
##   // group-level predictor values
##   vector[N] Z_1_1;
##   int prior_only; // should the likelihood be ignored?
## }
## transformed data {
##   matrix[N, Kc] Xc; // centered version of X without an intercept
##   vector[Kc] means_X; // column means of X before centering
##   for (i in 2:K) {
##     means_X[i - 1] = mean(X[, i]);
##     Xc[, i - 1] = X[, i] - means_X[i - 1];
##   }
## }
## parameters {
##   vector[Kc] b; // regression coefficients
##   real Intercept; // temporary intercept for centered predictors
##   real<lower=0> sigma; // dispersion parameter
##   vector<lower=0>[M_1] sd_1; // group-level standard deviations
##   array[M_1] vector[N_1] z_1; // standardized group-level effects
## }
## transformed parameters {
##   vector[N_1] r_1_1; // actual group-level effects
##   real lprior = 0; // prior contributions to the log posterior
##   r_1_1 = (sd_1[1] * (z_1[1]));
##   lprior += student_t_lpdf(Intercept | 3, 197, 16.3);
##   lprior += student_t_lpdf(sigma | 3, 0, 16.3)
##     - 1 * student_t_lccdf(0 | 3, 0, 16.3);
##   lprior += student_t_lpdf(sd_1 | 3, 0, 16.3)
##     - 1 * student_t_lccdf(0 | 3, 0, 16.3);
## }
## model {
##   // likelihood including constants
##   if (!prior_only) {
##     // initialize linear predictor term
##     vector[N] mu = rep_vector(0.0, N);
##     mu += Intercept;
##     for (n in 1:N) {
##       // add more terms to the linear predictor
##       mu[n] += r_1_1[J_1[n]] * Z_1_1[n];
##     }
##     target += normal_id_glm_lpdf(Y | Xc, mu, b, sigma);
##   }
}

```

```
## // priors including constants
## target += lprior;
## target += std_normal_lpdf(z_1[1]);
## }
## generated quantities {
## // actual population-level intercept
## real b_Intercept = Intercept - dot_product(means_X, b);
## }
```

Convergence The model fit procedure seems to have converged to a stable solution: $\hat{R}$ values are at 1, the number of effective sample sizes all lie above 5021, the posterior distributions are unimodal and relatively smooth, and the trace plots suggest that the chains mixed well (they lie on top of one another) showing a horizontal zig-zag like a “hairy caterpillar” with no systematic upwards or downwards drift.

Parameter estimates The model parameters have average values that roughly seem to make sense. For example, at 151.64 mm the mean of the intercept can reasonably be conceived as corresponding to the flipper length of a very small penguin (of beak length zero). An increase in bill length by one mm is estimated to go along with a corresponding increase in flipper length by 1.09 mm on average, though the random intercept of 8.79 indicates that the relationship between bill and flipper length varies between species.

All parameters appear large enough to contribute to the model’s ability to predict the observed values of the criterion variable (i.e. flipper length). For example, the 95% credible interval for the fixed effect of bill length [0.88, 1.29] is positive and does not include zero.

Model comparison

Until now we have only considered a relatively simple model with a fixed predictor for bill length, and a random intercept for species. We can consider complexer models with more predictors, as well as a minimal intercept-only model, and compare these to one another using Bayes factors to gain a better overview of which model specification offers the best balance between the ability to account for the pattern in the data and doing so without being unnecessarily complex resp. flexible. For instance, we can consider the following four models and compare them with the model we just computed.

- A random intercept model: `m_rintercept <- brm(flipper_len ~ 1 + (1 | species))`
- A model with a fixed predictor for bill depth instead of bill length: `m2 <- brm(flipper_len ~ bill_dep + (1 | species))`
- A model with both fixed predictors (without interaction): `m3 <- brm(flipper_len ~ bill_len + bill_dep + (1 | species))`
- A model with both fixed predictors and their interaction: `m4 <- brm(flipper_len ~ bill_len*bill_dep + (1 | species))`

To compute models that contain interacting predictor variables, we center the variables around their mean first in order to avoid problems of multicollinearity.

```
penguins$bill_len_c <- penguins$bill_len - mean(penguins$bill_len, na.rm = T)
penguins$bill_dep_c <- penguins$bill_dep - mean(penguins$bill_dep, na.rm = T)
```

```
output <- capture.output(
m_rintercept <- brm(flipper_len ~ 1 + (1 | species)
, data = penguins
, family = gaussian()
, sample_prior = TRUE
, warmup = 1000
, iter = 14000
, chains = 4
, cores = 4
```

```

    , save_pars = save_pars(all = TRUE)
    , control = list(adapt_delta = 0.99, max_treedepth = 12))
)

# again with more samples for bfs
output <- capture.output(
m1 <- brm(flipper_len ~ bill_len + (1 | species)
    , data = penguins
    , family = gaussian()
    , prior = prior_2
    , sample_prior = TRUE
    , warmup = 1000
    , iter = 4000
    , chains = 4
    , cores = 4
    , save_pars = save_pars(all = TRUE)
    , control = list(adapt_delta = 0.99, max_treedepth = 12))
)

output <- capture.output(
m2 <- brm(flipper_len ~ bill_dep + (1 | species)
    , data = penguins
    , family = gaussian()
    , sample_prior = TRUE
    , warmup = 1000
    , iter = 14000
    , chains = 4
    , cores = 4
    , save_pars = save_pars(all = TRUE)
    , control = list(adapt_delta = 0.99, max_treedepth = 12))
)

output <- capture.output(
m3 <- brm(flipper_len ~ bill_len + bill_dep + (1 | species)
    , data = penguins
    , family = gaussian()
    , sample_prior = TRUE
    , warmup = 1000
    , iter = 14000
    , chains = 4
    , cores = 4
    , save_pars = save_pars(all = TRUE)
    , control = list(adapt_delta = 0.99, max_treedepth = 12))
)

# Use the centered predictors
# Note that after centering, the intercept corresponds to
# the grand mean of the criterion variable "flipper length"
output <- capture.output(
m4 <- brm(flipper_len ~ bill_len_c*bill_dep_c + (1 | species)
    , data = penguins
    , family = gaussian()
    , sample_prior = TRUE
    , warmup = 1000

```

```

, iter = 14000
, chains = 4
, cores = 4
, save_pars = save_pars(all = TRUE)
, control = list(adapt_delta = 0.99, max_treedepth = 12))
)

```

Now we can compare the models using Bayes factors, in this case with the bridge sampling method.

```

# Marginal likelihoods as starting point
mar_rintercept <- bridge_sampler(m_rintercept, silent = T)
mar_m1 <- bridge_sampler(m1, silent = T)
mar_m2 <- bridge_sampler(m2, silent = T)
mar_m3 <- bridge_sampler(m3, silent = T)
mar_m4 <- bridge_sampler(m4, silent = T)

# Forward selection procedure
bf_1i <- bayes_factor(mar_m1, mar_rintercept) # > 100
bf_2i <- bayes_factor(mar_m2, mar_rintercept) # > 100
bf_3i <- bayes_factor(mar_m3, mar_m1)         # > 100
bf_32 <- bayes_factor(mar_m3, mar_m2)         # > 100
bf_43 <- bayes_factor(mar_m4, mar_m3)         # 0.277

# These two are equivalent
bf_34 <- bayes_factor(mar_m3, mar_m4) # 3.61
1/bf_43$bf
## [1] 3.568514

```

The *BF* comparison yielded that model *m3*, with main effects for bill length and bill depth, but without their interaction, achieved the best balance between accounting for the pattern in the data without being unnecessarily complex.

Posterior predictive check

```

set.seed(1234)

# Prior predictive check:
pp_check(m3)

# pp_check(m_prior_2, type='stat', stat='mean')

```

There is a good overall correspondence between model predictions and the pattern in the data.

Summary output for final model

Now that we have settled on *m3* as the relative best model among those we compared, we can visualise in a bit further detail its posterior parameter estimates, e.g. through plots of posterior distributions and tables with the most important numerical values. There are many options in R to do so - here we use the *bayesplot* package for graphics and the *knitr* package for tables.

General effects

```

color_scheme_set("orange") # plot the distributions in orange

# For the two fixed effects
g2 <- mcmc_areas(m3,
  pars = c("b_bill_len", "b_bill_dep"),

```

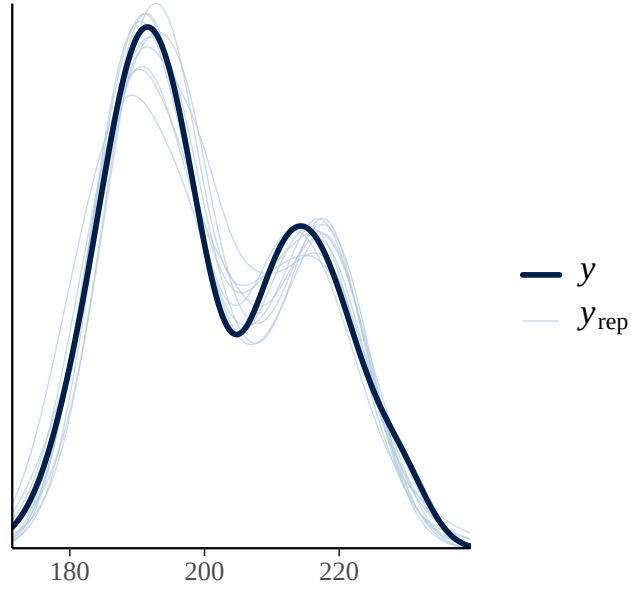


Figure 4: Posterior predictive check.

```

prob = 0.95, # 95% credible intervals
point_est = "mean"
) +
theme_bw() +
labs(x = "Estimate", y = "Parameter") +
scale_y_discrete(labels = c(expression(~italic(b[~"bill_length"])),
                             expression(~italic(b[~"bill_depth"]))))

```

g2

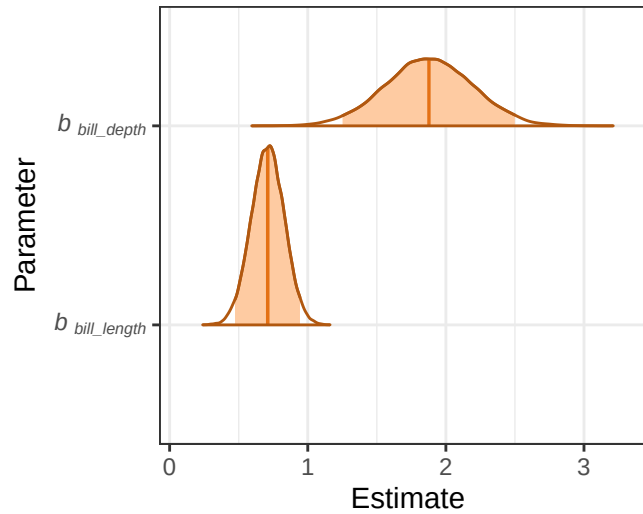


Figure 5: Population level (fixed) posterior parameter estimates for model 3.

```

m3_fixed <- summary(m3)[[15]][,c("Estimate", "Est.Error",
                                "l-95% CI", "u-95% CI", "Bulk_ESS", "Rhat")]
m3_fixed <- rownames_to_column(m3_fixed, var = "Parameter")
colnames(m3_fixed) <- c("Parameter", "Mean", "SE",
                       "Lower Bound", "Upper Bound", "$n_{eff}$", "$\\hat{R}$")

```

```
kable(m3_fixed, caption = "Fixed effects for model m3.")
```

Table 1: Fixed effects for model m3.

Parameter	Mean	SE	Lower Bound	Upper Bound	n_{eff}	$\hat{R}$
Intercept	135.6923596	10.3052880	114.8483279	156.0913904	20615.57	1.000125
bill_len	0.7103135	0.1195537	0.4733551	0.9437115	30279.18	1.000030
bill_dep	1.8776985	0.3183170	1.2518274	2.4994960	30431.25	1.000027

Individual effects (in this case just random intercept by species)

```
# Extract the parameter estimates
intercept_byspecies <- coef(m3)$species[,,"Intercept"][,c("Estimate", "Q2.5", "Q97.5")]
intercept_byspecies <- as.data.frame(intercept_byspecies)

# Extract the general effect estimate of the parameter
b_Intercept <- fixef(m3)["Intercept","Estimate"]

# Order the estimates ascending
intercept_byspecies2 <- intercept_byspecies[order(intercept_byspecies$Estimate),]
intercept_byspecies2$Order <- as.integer(c(1:3))

# Add parameter to dataframe. When the credible interval
# contains zero, the parameter is set pink, otherwise to blue
for (i in 1:nrow(intercept_byspecies2)){
  if (intercept_byspecies2$Q2.5[i]*intercept_byspecies2$Q97.5[i] < 0) {
    intercept_byspecies2$zero2[i] <- "#F8766D"
  } else (intercept_byspecies2$zero2[i] <- "#00BFC4")
}

# Plot the individual estimates for each parameter
g_intercept_byspecies2 <- ggplot(intercept_byspecies2,
  aes(x = rownames(intercept_byspecies2),
    y = Estimate,
    ymin = Q2.5,
    ymax = Q97.5)) +
  geom_linerange(color = intercept_byspecies2$zero2) +
  geom_point() +
  geom_hline(yintercept = 0, linetype = "dashed", alpha = .3) +
  geom_hline(yintercept = b_Intercept, color = "red", alpha = .3) +
  theme(legend.position = "none",
  axis.title.x=element_blank(),
  axis.text.x=element_blank(),
  axis.ticks.x=element_blank(),)+
  theme_bw()+
  labs(x = "Species")
g_intercept_byspecies2

m3_rintercept <- summary(m3)[[18]][[1]][,c("Estimate", "Est.Error",
  "1-95% CI", "u-95% CI",
  "Bulk_ESS", "Rhat")]

m3_residual <- summary(m3)[[16]][,c("Estimate", "Est.Error",
  "1-95% CI", "u-95% CI",
  "Bulk_ESS", "Rhat")]
```

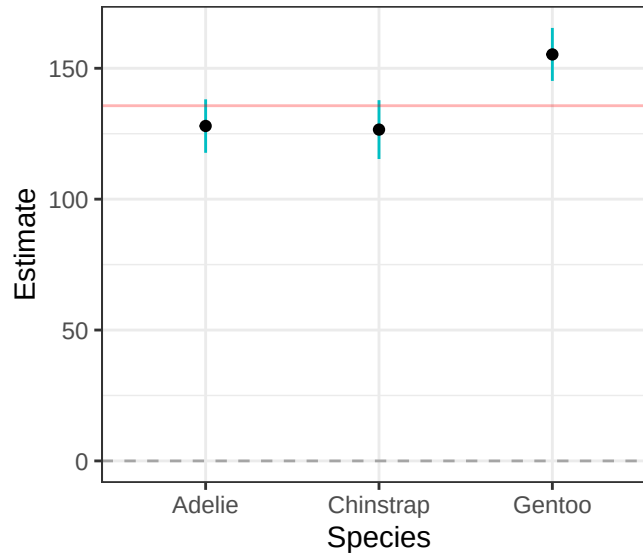


Figure 6: Random intercepts by species for model m3.

```
m3_rintercept <- rownames_to_column(m3_rintercept, var = "Parameter")
m3_residual <- rownames_to_column(m3_residual, var = "Parameter")
m3_rand <- rbind(m3_rintercept, m3_residual)
colnames(m3_rand) <- c("Parameter", "Mean", "SE",
  "Lower Bound", "Upper Bound",
  "$n_{eff}$", "$\\hat{R}$")
kable(m3_rand, caption = "Random effects for model m3.")
```

Table 2: Random effects for model m3.

Parameter	Mean	SE	Lower Bound	Upper Bound	n_{eff}	$\hat{R}$
sd(Intercept)	18.623862	8.7867590	8.259821	41.028893	18386.94	1.000081
sigma	5.559558	0.2148251	5.157019	6.001841	37748.31	1.000287

Extension

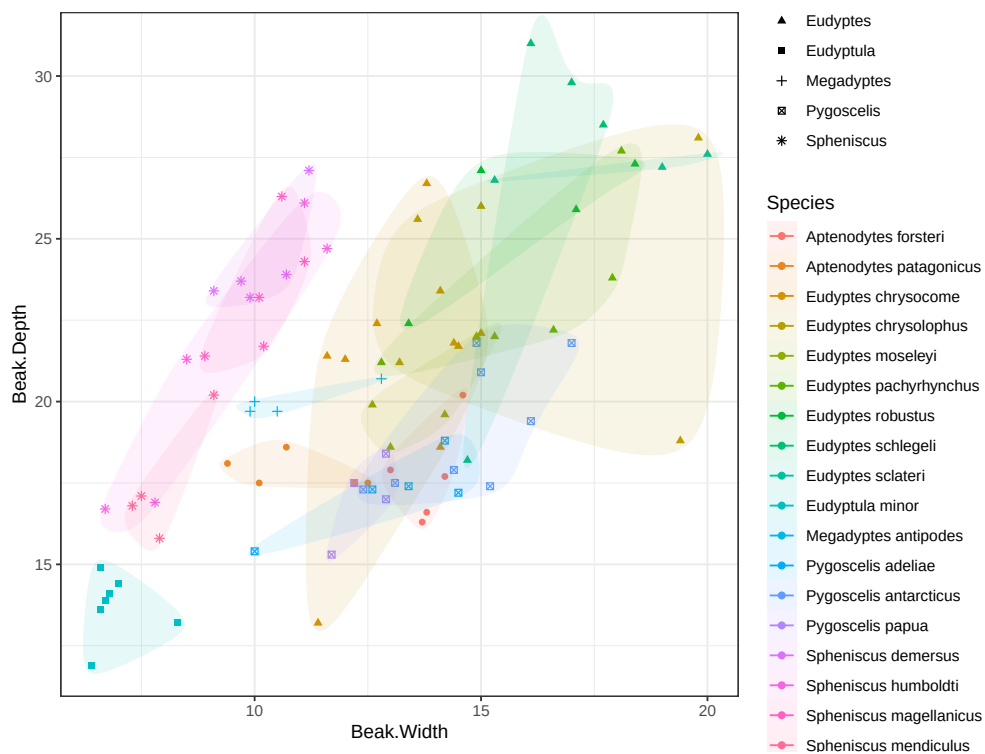
Ben Bolker shared a larger penguin dataset that is great for multilevel modelling because it features many more species, along with further penguin measurements. The inclusion of more species makes it easier to compute not just the overall variability of the criterion variable between species (random intercept), but also to what extent the relationship between predictors and criterion varies between species (random slopes). The github repository for the course contains further details. Dataset source: Tobias, Joseph A., Catherine Sheard, Alex L. Pigot, et al. 2022. “AVONET: Morphological, Ecological and Geographical Data for All Birds.” *Ecology Letters* 25 (3): 581–97. <https://doi.org/10.1111/ele.13898>.

```
pp <- readRDS("penguin_traits.rds")
head(data.frame(pp))
```

```
##           Species      genus Sex Age Locality Country
## 1 Aptenodytes forsteri Aptenodytes U  0      NA      NA
## 2 Aptenodytes forsteri Aptenodytes M  0      NA      NA
## 3 Aptenodytes forsteri Aptenodytes M  0      NA      NA
## 4 Aptenodytes forsteri Aptenodytes F  0      NA      NA
## 5 Aptenodytes forsteri Aptenodytes F  0      NA      NA
## 6 Aptenodytes patagonicus Aptenodytes M  0      NA      NA
```

##	Beak.Length_Culmen	Beak.Length_Nares	Beak.Width	Beak.Depth	Tarsus.Length
## 1	66.8	51.5	13.0	17.9	NA
## 2	102.4	47.6	14.2	17.7	46.7
## 3	103.7	59.6	13.7	16.3	45.7
## 4	94.4	57.5	14.6	20.2	50.8
## 5	103.7	51.2	13.8	16.6	45.8
## 6	102.1	67.2	12.2	17.5	51.7

##	Wing.Length	Kipps.Distance	Secondary1	Hand.wing.Index	Tail.Length
## 1	170	NA	NA	NA	111.0
## 2	185	5.1	179.9	2.8	119.0
## 3	180	6.2	173.8	3.4	120.0
## 4	182	8.0	174.0	4.4	130.0
## 5	180	7.5	172.5	4.2	107.5
## 6	166	7.1	158.9	4.3	62.0



For this dataset, a model predicting beak depth from beak width - while accounting for differences between species - could look like `m5` below. Since we now have more species. Using Bayes factors, we can compare this to a complexer model `m6` that includes a random slope for beak width alongside the random intercept.

```
output <- capture.output(
m5 <- brm(Beak.Depth ~ Beak.Width + (1 | Species)
, data = pp
, family = gaussian()
, sample_prior = TRUE
, warmup = 1000
, iter = 14000
, chains = 4
, cores = 4
, save_pars = save_pars(all = TRUE)
, control = list(adapt_delta = 0.99, max_treedepth = 12))
```

```

)

output <- capture.output(
m6 <- brm(Beak.Depth ~ Beak.Width + (Beak.Width | Species)
  , data = pp
  , family = gaussian()
  , sample_prior = TRUE
  , warmup = 1000
  , iter = 14000
  , chains = 4
  , cores = 4
  , save_pars = save_pars(all = TRUE)
  , control = list(adapt_delta = 0.99, max_treedepth = 12))
)

mar_m5 <- bridge_sampler(m5, silent = T)
mar_m6 <- bridge_sampler(m6, silent = T)
bf_65 <- bayes_factor(mar_m6, mar_m5) # 0.11479
1/bf_65$bf
## [1] 9.407549

```

In this case model m5 without the assumption that the relationship between beak width and beak depth varies between species was around 9.4 times more likely than m6 given the data. The variability in the beak width - beak depth relation between species may be too small to warrant the increased model complexity that would result from taking it into account.